HS PF

Programmieren in Python
Kontrollstrukturen

Prof. Dr.-Ing. Stefan Kray

Kontrollstrukturen

- Kontrollstrukturen sind Anweisungen, die den Programmablauf steuern
- Dazu zählen:
  - IF-Abfragen
  - Bedingte Ausdrücke
  - Schleifen
    - for-Schleife
    - while-Schleife
  - Try-Except Blöcke

IF-Abfragen (1)

- Einfachste Variante: if <Ausdruck>:

```
if True:
    print("Hallo")
```

- Achtung: Einrückung mit 4 Spaces (TAB wird in Spyder auf 4 Leerzeichen umgesetzt)

- IF-Else:

```
a=input()
if int(a)>=0:
    print("Die Zahl {0} ist größer gleich Null.".format(a))
else:
    print("Die Zahl {0} ist kleiner Null.".format(a))
```

- Die Funktion „format“ ersetzt die nummerierten Klammerausdrücke im String!

IF-Abfragen (2)

- IF-ELIF-ELSE:
  - Konstrukt mit else if:

```
a=input()
if int(a)>0:
    print("Die Zahl {0} ist größer Null.".format(a))
elif int(a)==0:
    print("Die Zahl {0} ist gleich Null.".format(a))
else:
    print("Die Zahl {0} ist kleiner Null.".format(a))
```

Switch?

- In C/C++ gibt es Switch-Abfragen:

```
switch(i) {
    case 1: cout << “1";
    break;
    case 2: cout << “2";
    break;
    default: cout << “Default";
}
```

- Python:

```
if i==1:
    print("1")
elif i==2:
    print("2")
else:
    print("default")
```

Switch (2)

- alternativ mit Dictionary:

```
cases={1:"Fall 1",2:"Fall 2"}
if i in cases:
    print(cases[i])
else:
    print("default")
```

Switch (3)

- seit Python 3.10: Befehl "match"

```
i=2
match i:
    case 1:
        print("1")
    case 2:
        print("2")
    case _:
        print("default")
```

IF-Ausdrücke

- IF-ELSE auch in einer Zeile möglich
- <Wert_True> if <Bedingung> else <Wert_False>

- Beispiel:

```
b=20 if a==1 else 10
```

  - Wenn a==1 erhält b den Wert 20
  - Sonst 10

```
print("Ja" if 1==0 else "Nein")
```

Aufgabe

- Schreiben Sie ein Programm „gerade.py“ **in einer Zeile**, dass:
  - Eine Zahl vom Benutzer einliest
  - Ausgibt, ob die Zahl gerade oder ungerade ist

FOR-Schleife (1)

- Die FOR-Schleife ist die gängigste Schleife in Python
- Sie benötigt eine Variable und ein Range-Objekt
- for <variable> in <range_objekt>:
- mit Einrückung dann den eigentlichen Schleifenkörper
- Beispiel:

```
for i in range(0,5):
    print(i)
```

Ergebnis: 0,1,2,3,4

FOR-Schleife (2)

- Die FOR-Schleife hat auch ein Else-Konstrukt
- Dieses wird nur ausgeführt, wenn die Schleifenausführung nicht abgebrochen wird

```
for i in range(0,5):
    print(i)
else:
    print("Ende")
```

Ergebnis: 0,1,2,3,4,“Ende“

FOR-Schleife (3)

- Die FOR-Schleife hat auch ein Else-Konstrukt
- Schleife kann mit „break“ abgebrochen werden

```
for i in range(0,5):
    if (i==3):
        break
    print(i)
else:
    print("Ende")
```

- **Ergebnis ?**

FOR-Schleife (4)

- Die FOR-Schleife hat auch ein Else-Konstrukt
- Iteration kann mit „continue“ übersprungen werden

```
for i in range(0,5):
    if (i==3):
        continue
    print(i)
else:
    print("Ende")
```

- **Ergebnis 0,1,2,4,Ende**

FOR-Schleife

- Bei vielen Datentypen „Abkürzungen“ möglich:

```
liste=[1,2,3]
for l in liste:
    print(l)
```

Ergebnis: 1,2,3

```
for s in "Hallo":
    print(s)
```

Ergebnis: H,a,l,l,o

Aufgabe:

- Berechnen Sie die Inzidenz für **alle Werte** der Variable ncases in dem Programm „corona.py“. Benutzen Sie eine for-Schleife. Plotten Sie Indzidenz mit Hilfe des Befehls „plot“ (unten im code).

- Inzidenz0=$\frac{\#Tag0+\#Tag1+\#Tag2+\#Tag3+\#Tag4+\#Tag5+\#Tag6}{Einwohnerzahl} * 100.000$
- Inzidenz1=$\frac{\#Tag1+\#Tag2+\#Tag3+\#Tag4+\#Tag5+\#Tag6+\#Tag7}{Einwohnerzahl} * 100.000$
- ...
- Inzidenz_n=$\frac{\#Tag_n+\#Tag_{n+1}+\#Tag_{n+2}+\#Tag_{n+3}+\#Tag_{n+4}+\#Tag_{n+5}+\#Tag_{n+6}}{Einwohnerzahl} * 100.000$

```
import matplotlib.pyplot as plt
plt.plot(filtered)
plt.xlabel('Tag der Pandemie')
plt.ylabel('7 Tage Inzidenz')
```

WHILE-Schleife (1)

- Syntax: while <bedingung>:
- Abbruch: Wenn Bedingung nicht mehr erfüllt oder durch break
- Beispiel:

```
print("Bitte Zahl eingeben. a==0,a<0 --> Ende")
a=1
while int(a)>0:
    a=input()
    if int(a)==0:
        print("a==0 --> Ende")
        break
    else:
        print("a ist {0}".format(a))
print("Ende")
```

- **Ergebnis bei a=13,a=0,a=-12,a=3.3?**

WHILE-Schleife (2)

- Endlosschleife:

```
while True:
    pass
```

- Ausdruck „pass“: → nichts tun

Try-Except-Blöcke

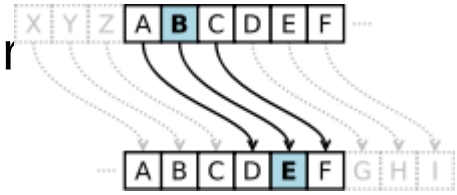
- Wie erkennt man Fehler zur Laufzeit?
- Lösung: Try-Except-Blöcke
- Beispiel:

```
try:
    num=int(input("Geben Sie eine ganzzahlige Zahl ein:\n"))
    print(num)
except Exception as e:
    print("Error:",e)
```

- Statt der allgemeinen Exception können diese auch genauer definiert und unterschieden werden, wie z.B. ValueError, etc. Der except-Teil kann dann in mehrere Äste aufgespalten werden, ähnlich zu elif in if-Abfragen.

Aufgabe „decrypt.py“: Knacken Sie den Code!

- Ein Text (aus einem Kriminalfall) wurde per Caesar-Algorithmus verschlüsselt
- Jeder Buchstabe wird um einen konstanten Wert „verschoben“
- Dieser konstante Wert ist Ihnen nicht bekannt
- Erweitern Sie das Programm „decrypt.py“ um eine Entschlüsselung
- Entschlüsseln Sie die Variable **encrypted1: mDlstiD}syDeviDlezmrkDpsxwDsjDjyr**
- Hinweise:
  - Die Funktion ord() liefert den Ascii-Code zu einem Zeichen
    - Beispiel: ord(“a“) → 97
  - Die Funktion chr() verwandelt den Ascii-Wert einer wieder in einen String Beispiel: chr(97) → “a“
  - Strings addieren: `s="a"+"b"` → `"ab"`



Quelle: https://de.wikipedia.org/wiki/Caesar-Verschl%C3%BCsselung#/media/Datei:Caesar3.svg

ASCII-Code-Tabelle

32	SP	64	@	96	`	128	Ç	160	á	192	└	224	Ó
33	!	65	A	97	a	129	ü	161	í	193	┴	225	ß
34	"	66	B	98	b	130	é	162	ó	194	┬	226	Ô
35	#	67	C	99	c	131	â	163	ú	195	├	227	Ò
36	$	68	D	100	d	132	ä	164	ñ	196	─	228	õ
37	%	69	E	101	e	133	à	165	Ñ	197	┼	229	Õ
38	&	70	F	102	f	134	å	166	ª	198	ã	230	µ
39	'	71	G	103	g	135	ç	167	º	199	Ã	231	þ
40	(	72	H	104	h	136	ê	168	¿	200	╚	232	Þ
41	)	73	I	105	i	137	ë	169	®	201	╔	233	Ú
42	*	74	J	106	j	138	è	170	¬	202	╩	234	Û
43	+	75	K	107	k	139	ï	171	½	203	╦	235	Ù
44	,	76	L	108	l	140	î	172	¼	204	╠	236	ý
45	-	77	M	109	m	141	ì	173	¡	205	═	237	Ý
46	.	78	N	110	n	142	Ä	174	«	206	╬	238	¯
47	/	79	O	111	o	143	Å	175	»	207	¤	239	´
48	0	80	P	112	p	144	É	176	░	208	ð	240	-
49	1	81	Q	113	q	145	æ	177	▒	209	Ð	241	±
50	2	82	R	114	r	146	Æ	178	▓	210	Ê	242	‗
51	3	83	S	115	s	147	ô	179	│	211	Ë	243	¾
52	4	84	T	116	t	148	ö	180	┤	212	È	244	¶
53	5	85	U	117	u	149	ò	181	Á	213	ı	245	§
54	6	86	V	118	v	150	û	182	Â	214	Í	246	÷
55	7	87	W	119	w	151	ù	183	À	215	Î	247	¸
56	8	88	X	120	x	152	ÿ	184	©	216	Ï	248	°
57	9	89	Y	121	y	153	Ö	185	╣	217	┘	249	¨
58	:	90	Z	122	z	154	Ü	186	║	218	┌	250	·
59	;	91	[	123	{	155	ø	187	╗	219	█	251	¹
60	<	92	\	124	|	156	£	188	╝	220	▄	252	³
61	=	93	]	125	}	157	Ø	189	¢	221	¦	253	²
62	>	94	^	126	~	158	×	190	¥	222	Ì	254	■
63	?	95	_	127	DEL	159	ƒ	191	┐	223	▀	255	

Aufgabe „decrypt.py“, Teil 2

- Die Variable encrypted2 enthält einen weiteren verschlüsselten String:
- es wurde folgender Algorithmus zur Verschlüsselung verwendet:

```
encrypted2=""
i=0
for s in eingabe:
    encrypted2+=chr(ord(s)+i % mod+32)
    i=i+1
```

Die Variable „mod“ enthält eine Ihnen unbekannte Zahl

Entschlüsseln Sie den String in **encrypted2: ioBwvyjpjDtpBfetdjCqe** indem Sie die Eingabe rekonstruieren.